FleetFlow

Лабораторна робота №8

Документування та захист архітектури

Варіант 12: управління автопарком та маршрутами

Підсумковий пакет наскрізного проєкту ЛР1-ЛР8

Код, архітектурні рішення, результати перевірок і план еволюції

03 листопада 2026 року

Репозиторій: github.com/VladyslavCh412/labs · ветка lab8-complete

Зміст

1. Огляд системи та межі 3

2. Глосарій 4

3. C4: контекст, контейнери, компоненти 5

4. Архітектурні рішення та еволюція 8

5. Чотири шари та бізнес-правила 9

6. Патерни та підтвердження рефакторингу 10

7. Архітектура даних 11

8. Безпека та модель загроз 12

9. Сценарії якості з ЛР1 13

10. Тести, контракти та конвеєр 16

11. Прийнятий технічний борг 17

12. План еволюції на три кроки 18

13. Висновок 19

Джерела 20

Додаток A. Аудит і докази 21

Додаток B. Демонстрація застосунку 22

Додаток C. Повний журнал ADR 23

Додаток D. Патерни: діаграми до і після 27

Observer: продуцент та підписники 28

1. Огляд системи та межі

FleetFlow обліковує автомобілі, водіїв, маршрути й рейси. Диспетчер створює рейс, починає перевезення і завершує його із зазначенням пробігу. Керівник керує автопарком. Спостерігач читає дані. Фактичні зовнішні актори - ці три ролі та браузер користувача. Картографія, GPS/телематика й зовнішні SMS/e-mail є частиною початкового задуму, але не підключені до цього прототипу.

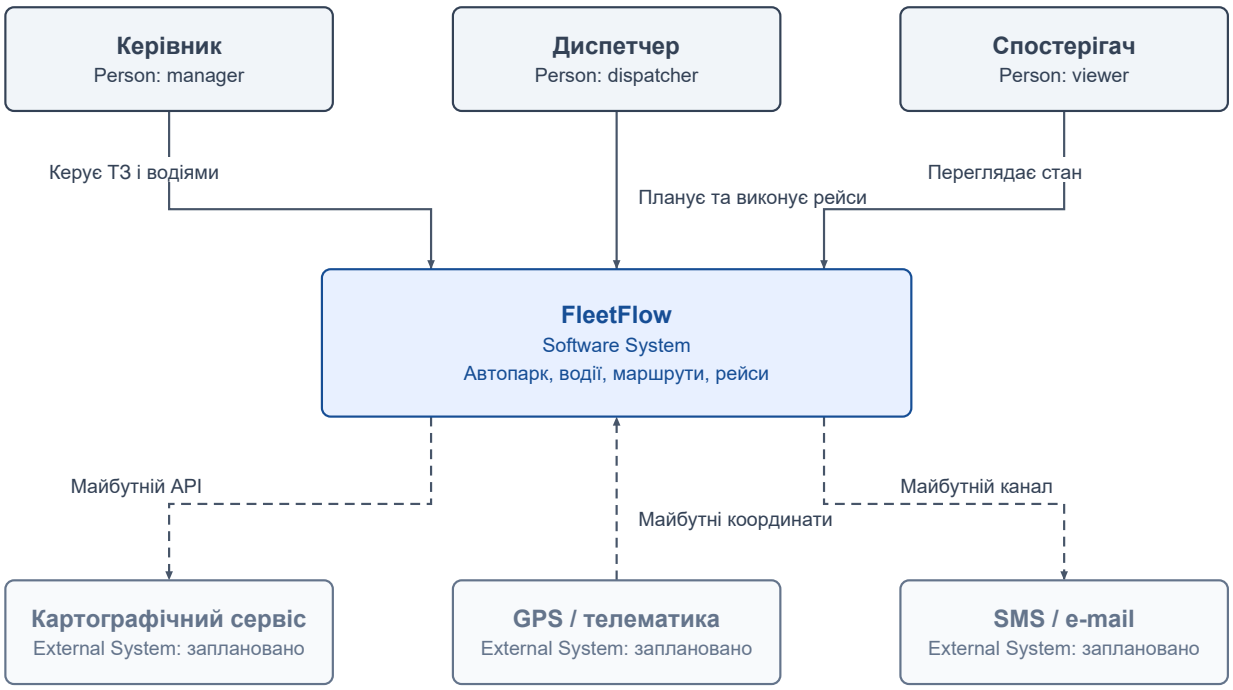
Зацікавлена сторона	Інтерес	Пріоритет
Диспетчер	Призначення та контроль рейсів	Швидкодія, актуальність
Водій	Коректне призначення та допуск	Цілісність, доступність
Керівник автопарку	Стан ТЗ і пробіг	Точність, безпека
Механік	Своєчасне ТО	Цілісність стану ТЗ
Адміністратор	Доступ, резервування, запуск	Безпека, відновлення
Бухгалтер	Облік використання транспорту	Точність пробігу
Замовник перевезень	Прозорий стан виконання	Надійність
Розробник	Передбачувані зміни	Підтримуваність

2. Глосарій

Термін	Значення
Автопарк	Сукупність транспортних засобів організації
Транспортний засіб (ТЗ)	Автомобіль з унікальним id, номером, станом і пробігом
Водій	Працівник, який має бути активним і мати дійсне посвідчення
Диспетчер	Користувач, який координує рейси
Рейс	Призначення ТЗ, водія й маршруту; planned → active → completed
Маршрут	Упорядкований перелік щонайменше двох непорожніх точок
Точка маршруту	Назва місця відвідування; геокодування не реалізовано
GPS-трек	Послідовність координат з часом; заплановано
Телематика	Дані бортових пристроїв; заплановано
Технічне обслуговування (ТО)	Стан maintenance, який забороняє початок рейсу
Простій	Час без перевезень; окрема аналітика запланована
Паливні витрати	Обсяг/вартість пального; заплановано
Геозона	Область контролю переміщення; заплановано
Сповіщення	Локальне повідомлення за подією TripCreated
Завантаженість	Використання автопарку; у панелі є кількість доступних ТЗ

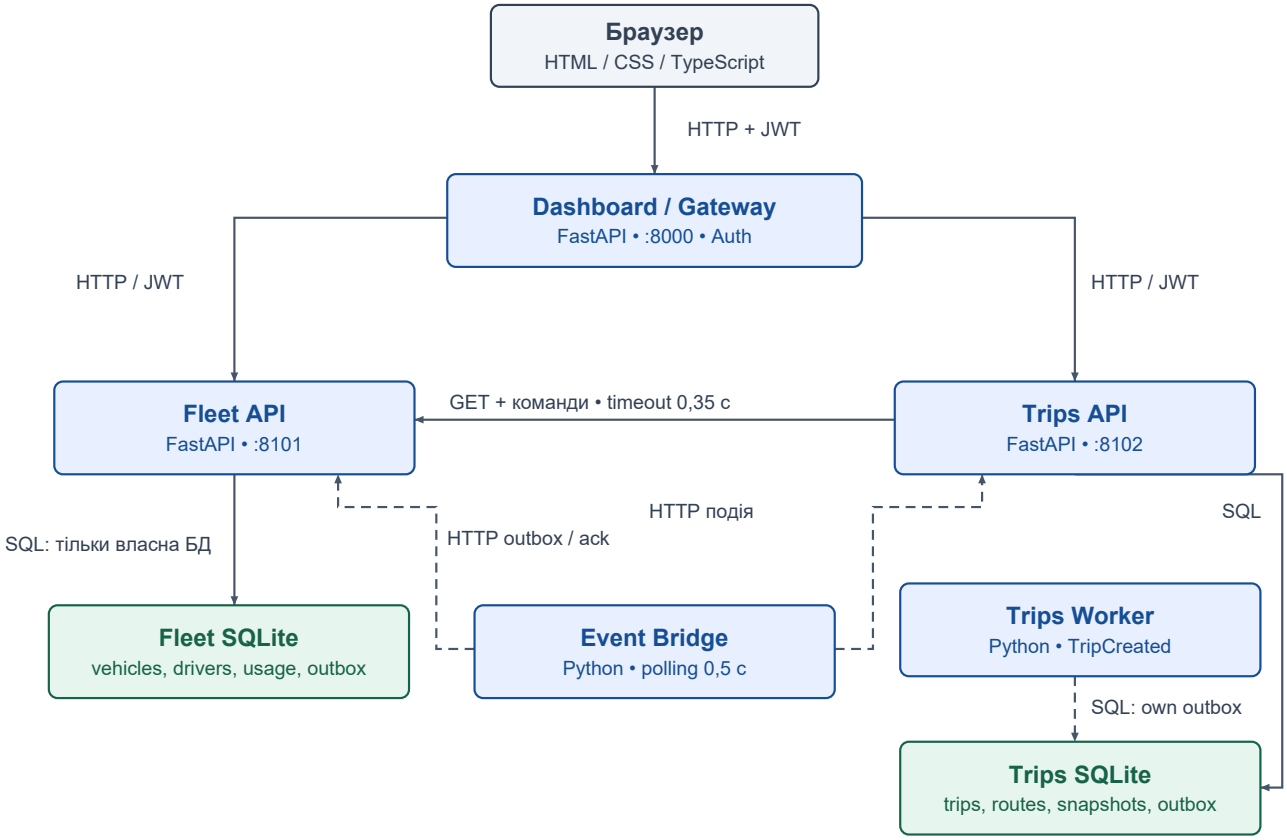
3. C4: контекст, контейнери, компоненти

C4 L1 - Контекст FleetFlow



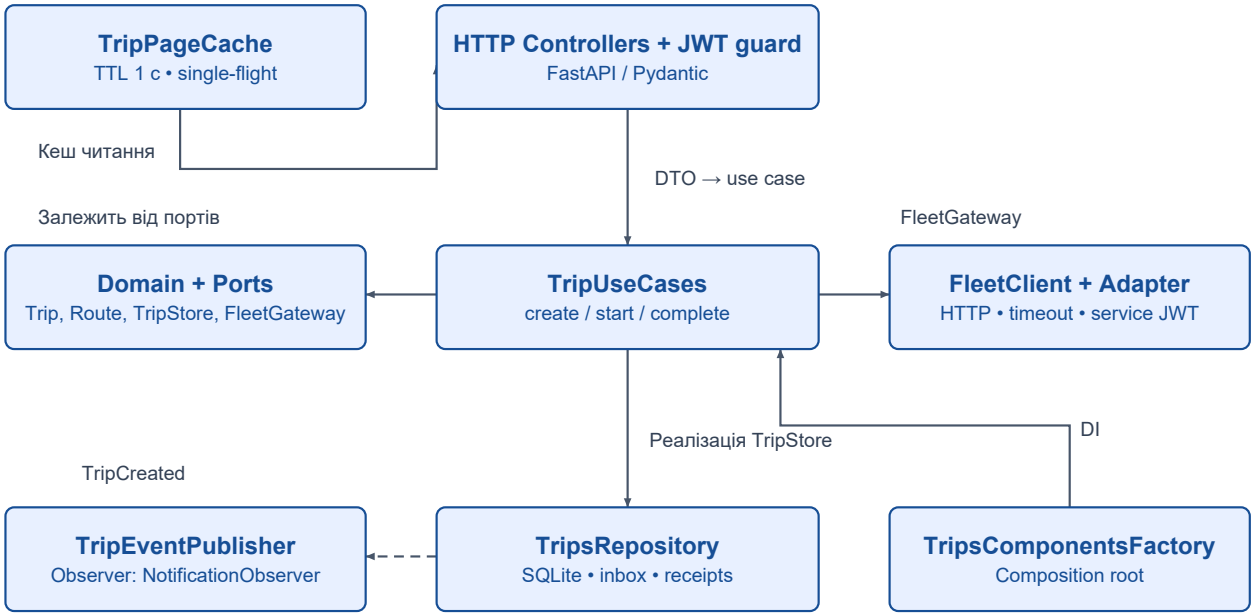
Легенда: синій - контейнер, сірий - актор/план, зелений - дані. Суцільний зв’язок - виклик, пунктир - подія або майбутня інтеграція.

C4 L2 - Контейнери й власники даних



Легенда: синій - контейнер, сірий - актор/план, зелений - дані. Суцільний зв’язок - виклик, пунктир - подія або майбутня інтеграція.

C4 L3 - Компоненти контейнера Trips



Легенда: синій - контейнер, сірий - актор/план, зелений - дані. Суцільний зв'язок - виклик, пунктир - подія або майбутня інтеграція.

Джерела діаграм: diagrams/context.mmd, containers.mmd, components.mmd; поряд збережені SVG. На L1 FleetFlow показано одним елементом. Суцільні зв'язки означають реалізовані HTTP/SQL-виклики; пунктирні - асинхронну доставку. Світлі елементи на L1 - зовнішні системи з плану.

Strategy викликається робочим Dashboard/Gateway через resolve_target; запит до неправильного власника ресурсу відхиляється.

L2: браузер звертається лише до Dashboard/Gateway (FastAPI, :8000). Fleet (:8101) володіє fleet.sqlite, Trips (:8102) - trips.sqlite. Event Bridge отримує outbox Fleet через HTTP, надсилає події Trips і підтверджує доставку лише після успіху. Trips Worker належить контексту Trips і читає тільки його outbox. Сервіси не читають чужі БД.

L3 деталізує тільки Trips: HTTP-контролери й JWT guard; TripUseCases; домени та порти TripStore/FleetGateway; SQLite repository; HTTP adapter; кеш сторінок; Observer-підписники. Factory збирає конкретні залежності. application.py імпортує тільки домен, що контролюється fitness-функцією.

Фронтенд реалізовано на TypeScript 7.0.2 у frontend/src/app.ts: strict і noEmitOnError, типізовані DTO та API-клієнт. npm run build створює runtime JS; FastAPI роздає тільки frontend/build. Node.js 24 використовується для збірки. Бекенд - FastAPI + SQLite. Джерело рішення - ADR-013.

4. Архітектурні рішення та еволюція

Початкове рішення - модульний моноліт (ADR-001); після ЛР3 deployment замінено Fleet + Trips (ADR-003). Технологічний стек FastAPI/SQLite збережено (ADR-002). Python у перевіреному середовищі - 3.13.14. SQLite обрано для простого локального стенду; незалежні процеси додають мережеві відмови.

Критерій	Модульний моноліт	Моноліт + worker	Fleet + Trips
Простота розробки/експлуатації	Найкраща: один deployment	Потрібен фоновий процес	Більше процесів і конфігурації
Ізоляція відмов	Спільний процес	Worker ізольований	Fleet/Trips мають незалежні збої
Цілісність	Одна локальна транзакція	Локальні транзакції та outbox	Лише локальний ACID, повтори й проєкції
Перевірюваність меж	Дисципліна імпортів	Додаткові тести worker	Контракти, тайм-аути, власник даних

Журнал з 14 рішень - docs/adr/INDEX.md. Оригінальні ADR-001-003 залишено незмінними. Індекс явно вказує заміну ADR-001 на ADR-003. Нові ADR-011/012/013 датовані 03.11.2026, не заднім числом.

5. Чотири шари та бізнес-правила

Поточний продукт містить усі чотири шари: presentation - trips_service/app.py; application - trips_service/application.py; domain - trips_service/domain.py (сутності та Protocol-порти); infrastructure - repository.py і fleet_client.py. Складання залежностей - factory.py та create_app. Сценарії create/start/complete звертаються до портів, а не конкретного SQLite. Vehicle і Driver належать домену Fleet, Trip і Route - домену Trips. HTTP DTO не проходять у SQLite напряму.

Правила: ідентифікатори рейсу обов'язкові; маршрут містить >=2 непорожні точки; пробіг не від'ємний; початок можливий лише для planned, доступного ТЗ і допущеного водія; ТО заборонено під час активного рейсу; завершення можливе лише для active. Завершення повертає ТЗ у available і додає пробіг.

Початок двох різних рейсів на одному ТЗ серіалізується BEGIN IMMEDIATE у Fleet. Квитанція {trip_id}:{operation} і зміна ТЗ комітяться разом. Повтор з іншим пробігом відхиляється 409. Trips записує власний перехід після успішної команди Fleet. При втраті відповіді або локальному збої той самий повтор безпечно завершує операцію. Автоматичний recovery worker для цих команд поки не реалізовано: це явний технічний борг.

6. Патерни та підтвердження рефакторингу

Проблема	Патерн і група	Альтернатива	Перевірений результат
Умови маршрутизації gateway зростають з endpoint	Strategy, поведінковий	Ланцюг if	Реєстр правил у gateway_routing.py, новий префікс додається правилом
Trips знає зовнішній JSON Fleet	Adapter, структурний	Звертатися до полів скрізь	FleetResponseAdapterізолює VehicleAvailability/DriverPermission
Збирання repo/client дублюється	Factory, породжувальний	Конструювання в контролерах	TripsComponentsFactory - одна точка складання; замінні фабрики в тестах
Продуцент знає кожного підписника	Observer, поведінковий	Прямі виклики споживачів	TripEventPublisher приймає список підписників, duplicate event_id не дублює notification

Характеризаційні твердження tests/test_characterization.py збережено. Історія: 46aa488 - тести; 42cb2c2 - Strategy; c99e7d3 - Adapter; f37eefe - Factory; 0492968 - Observer. Це наявна історія репозиторію, а не нові заднім числом створені коміти. Порівняння для кожного патерна: місце зміни при розширенні перейшло від контролера/продуцента до додавання окремого правила, адаптера, фабрики або підписника. Точні структурні показники до/після обчислюються AST-скриптом verification/pattern_metrics.py з включених before-файлів та поточного коду. Порівнюються гілки у proxy, прямі JSON-звернення споживача, пряме конструювання конкретних адаптерів у create_app та пряма SQL-залежність deliver_pending від notification_inbox. Це локальні метрики зв’язування, а не загальна оцінка складності системи. Результат: docs/evidence/pattern-metrics.json; джерела й історія: docs/refactoring/; дві діаграми до/після: diagrams/patterns/.

Патерн	Локальний показник	До	Після
Strategy	Кількість if у центральному proxy	2	1
Adapter	Прямі звернення до JSON полів vehicle/driver у споживачі	3	0
Factory	Прямі конструктори конкретних repo/client у create_app	2	0
Observer	Прямі SQL-вставки notification_inbox у продуцента	1	0

Порівняльні діаграми класів включено до diagrams/patterns/ і Додатка D PDF.

7. Архітектура даних

Сутність/таблиця	Власник	Споживач і спосіб доступу
Vehicle, Driver, vehicle_usage	Fleet	Trips: HTTP GET і внутрішня команда
command_receipts, vehicle_outbox	Fleet	Bridge: лише HTTP outbox/ack
Route, Trip, trip_operations	Trips	Dashboard: API Trips
vehicle_snapshot	Trips, похідна проєкція	Оновлюється лише подіями Fleet
notification_inbox, outbox	Trips	Окремий worker контексту Trips
Хеші облікових записів	Auth/bootstrap	Сервіси отримують claims JWT, не пароль

Локальна копія містить лише id, plate, status, version, updated_at. Вона потрібна для відображення останнього відомого стану; рішення про відправлення рейсу перевіряються у власника Fleet. Дані посвідчення водія та пробіг не копіюються. Міжсервісні посилання - id, без cross-DB FK.

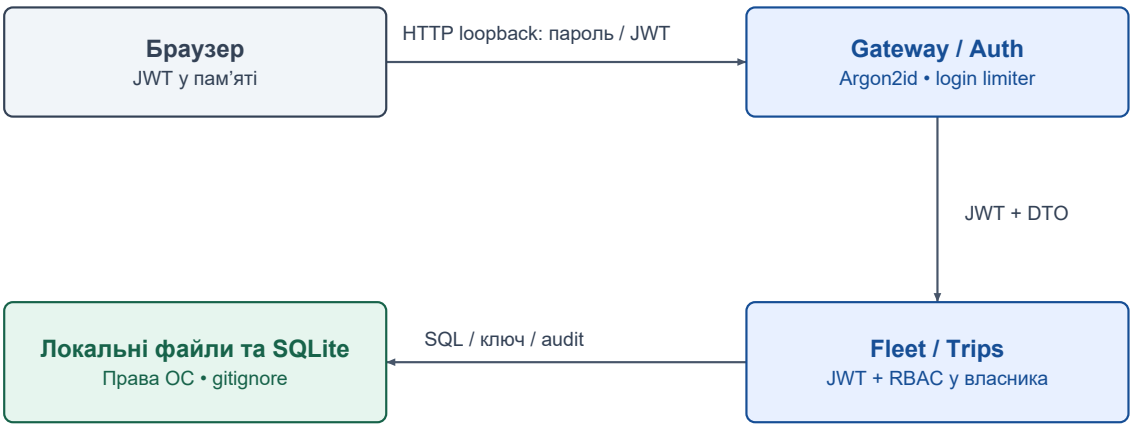
Міграція ЛР5: 001_expand додає registration_number; 002_dual_write підтримує паралельний запис; 003_switch_reads переводить читання; 004_contract вилучає plate тільки після переходу споживачів. Тест перевіряє кожен етап, доступ старого споживача до contract та повторний запуск. Адитивна feature-міграція fleet_service/lifecycle/001_usage.sql додає стан використання і квитанції, не змінюючи чотири історичні етапи.

Кеш	Ключ	TTL/місткість	Інвалідація
VehicleReadCache	vehicle_id	30 с	Зміна status, registration та lifecycle
TripPageCache	limit, offset, active	1 с, 128 ключів	Create, start, complete рейсу

Промах звертається до власного repository. Кеш сторінок має single-flight: конкурентні однакові запити не запускають кілька loader. Кеш локальний для процесу, тому інший процес бачить зовнішню зміну протягом TTL.

Fleet у тій самій транзакції з мутацією пише VehicleStatusChanged. Bridge повторює доставку при відмові й ack-ає тільки підтверджену подію. Trips у транзакції записує inbox event_id та оновлює snapshot тільки за більшої version. Дубль не змінює стан; старі події не відкочують нові. Номер автомобіля також породжує подію. Звичайний інтервал polling - 0,5 с; за outage верхньої межі відставання немає, що прийнятно для read-only проєкції, але не для рішення про відправлення ТЗ.

8. Безпека та модель загроз



Межі довіри: браузер → Gateway (пароль/JWT, DTO); Gateway → власник ресурсу (JWT, DTO); Trips → Fleet (service JWT, команда); Bridge → API обох сервісів (service JWT, подія); процес → власний SQLite/локальні файли. Усі сервери слухають лише loopback. Shared HS256 означає взаємну довіру локальних процесів; для незалежних зон довіри потрібні асиметричні ключі.

STRIDE-загроза	Вплив / імовірність	Захист і залишковий ризик
Підроблений/прострочений JWT	Високий / середня	HS256 allowlist, exp/iss/aud/iat/jti; 401
Прямий обхід gateway	Високий / висока	RBAC у Fleet і Trips; 403 у прямих запитах
Викрадення пароля/підбір	Високий / середня	Argon2id, випадкові паролі, 10 спроб/хв на адресу
Підміна внутрішньої команди	Високий / середня	Лише роль trips-service; людські ролі відхиляються
Повтор/перестановка подій	Середній / висока	inbox event_id + monotonic version
SQL injection / невалідний DTO	Високий / середня	Параметризований SQL, Pydantic, строгий пробіг
Заперечення виконаної операції	Середній / середня	Audit без паролів і JWT; адміністратор ОС може змінити файл
Витік секретів із Git	Високий / середня	data/, ключі, БД, облікові дані ігноруються Git
Відмова в обслуговуванні	Високий / середня	LIMIT<=100, bounded cache, timeout; глобальні квоти ще відсутні

JWT має sub, role, iss, aud, iat, exp, jti; строк дії - 900 с. manager: Fleet writes і Trips writes; dispatcher: Fleet reads і Trips writes; viewer: лише reads. bridge: outbox/ack Fleet та POST fleet-status Trips. trips-service: Fleet reads та POST internal/trip-commands. Людський manager не має доступу до внутрішньої команди. Дані токена не зберігаються у localStorage.

9. Сценарії якості з ЛР1

Нижче збережено всі вісім початкових сценаріїв з актуального локального lab1.docx, на який уже посилався аналіз репозиторію. Новими вимірюваннями не замінюється первинна ціль. Для нереалізованого функціоналу виконання не заявляється.

NFR-1: Продуктивність

Джерело: Диспетчер. Стимул: Відкриває список активних рейсів. Артефакт і середовище: API списку рейсів; Пікове навантаження 100 одночасних користувачів. Реакція: Система повертає першу сторінку з 50 рейсами. Міра: p95 часу відповіді ≤ 500 мс.

NFR-2: Продуктивність

Джерело: Диспетчер. Стимул: Запитує побудову маршруту. Артефакт і середовище: Інтеграція маршрутизації; Зовнішній сервіс доступний; маршрут до 20 точок. Реакція: Система показує результат або контрольовану помилку. Міра: 95% запитів завершуються ≤ 3 с.

NFR-3: Надійність

Джерело: GPS/телематична платформа. Стимул: Надсилає пакет координат. Артефакт і середовище: Приймання телеметрії; До 50 повідомлень/с. Реакція: Система зберігає валідні повідомлення без дублювання. Міра: ≥ 99,5% валідних повідомлень збережено; затримка ≤ 10 с.

NFR-4: Надійність

Джерело: Адміністратор. Стимул: Відбувається збій застосунку. Артефакт і середовище: Вебсистема та база даних; Звичайний робочий режим. Реакція: Система відновлює обслуговування після перезапуску. Міра: RTO ≤ 30 хв; RPO ≤ 5 хв.

NFR-5: Безпека

Джерело: Неавторизований користувач. Стимул: Намагається відкрити дані рейсів. Артефакт і середовище: Автентифікація та авторизація; Користувач не має активної сесії або відповідної ролі. Реакція: Доступ заборонено, подія записана в журнал. Міра: 100% таких запитів відхиляються; журнал містить подію ≤ 2 с.

NFR-6: Безпека

Джерело: Адміністратор. Стимул: Змінює роль користувача. Артефакт і середовище: Керування доступом; Адміністратор автентифікований. Реакція: Система перевіряє права та журналює зміну. Міра: 100% змін ролей мають запис аудиту.

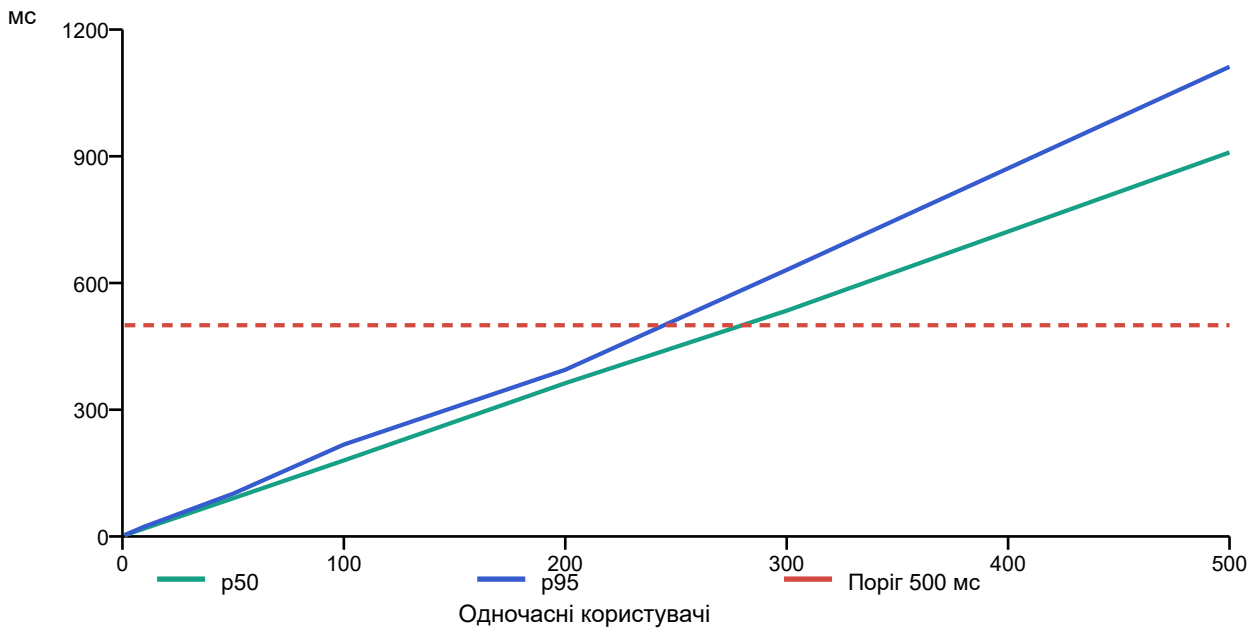
NFR-7: Підтримуваність

Джерело: Розробник. Стимул: Змінює правило нагадування про ТО. Артефакт і середовище: Модуль бізнес-правил; Є автоматизовані тести та CI. Реакція: Зміна не потребує редагування не пов’язаних модулів. Міра: Типова зміна реалізується ≤ 4 год; тести проходять.

NFR-8: Модифікованість/інтеграція

Джерело: Адміністратор. Стимул: Зовнішній сервіс сповіщень недоступний. Артефакт і середовище: Інтеграція сповіщень; Недоступність до 10 хв. Реакція: Основні операції з рейсами продовжують працювати; повідомлення повторюються. Міра: Основні API доступні ≥ 99% часу збою; повторні спроби починаються ≤ 1 хв.

Підсумок вимірювань



Режим	Users	p50, мс	p95, мс	Помилки/спроби
baseline	1	33.33	39.334	0/88
baseline	10	2555.763	2657.585	0/20
baseline	50	10971.984	13827.748	0/50
baseline	100	20263.21	20485.782	80/100
baseline	200	20805.533	20897.32	200/200
optimized	1	2.8	3.231	0/1056
optimized	10	19.008	23.353	0/1571
optimized	50	90.358	101.207	0/1655
optimized	100	179.934	217.53	0/1656
optimized	200	362.842	394.557	0/1690
optimized	300	534.414	630.933	0/1742
optimized	500	909.176	1111.889	0/1764

Окремий gate: p95 261.852 мс; 1461 відповідей; 0 помилок. Coverage домену 100%; 85 функціональних кейсів + 1 HTTP gate.

Числові результати і таблиця p50/p95 зберігаються в docs/evidence/. NFR-1 перевірено реальним HTTP gate при 100 користувачах; додатково виміряно 20 000 рейсів і рівні 1/10/50/100/200, а також 300/500 для пошуку межі. Оптимізація: SQL LIMIT/OFFSET та кеш сторінок замість fetchall з проєкцією всіх рядків. Server-Timing вимірює сегмент repository.

Сценарій	Фактичний стан	Висновок
NFR-1, p95 <=500 мс /100 users	Gate p95 261.852 мс, 1461 запитів, 0 помилок	Виконано локально; 300 users перевищують поріг
NFR-2, карти <=3 с	Немає зовнішнього routing API	Не реалізовано
NFR-3, GPS 50/s, >=99,5%, <=10 с	Немає ingest телеметрії	Не реалізовано
NFR-4, RTO<=30 хв/RPO<=5 хв	Немає backup/restore drill	Не підтверджено
NFR-5, заборона доступу й audit<=2 с	401/403 та audit у тому самому виклику тестуються	Підтверджено для тестових кейсів; універсальний відсоток не доведений
NFR-6, аудит змін ролей	Немає endpoint керування ролями	Не реалізовано
NFR-7, зміна ТО<=4 год	Pure domain, тести й CI є; час людської зміни не виміряно	Підтримано структурно, не підтверджено за часом
NFR-8, зовнішні сповіщення outage10 хв	Локальний outbox перевірено; зовнішній канал відсутній	Частково, 10-хвилинний експеримент не проводився

План навантаження: один Uvicorn worker, loopback, SQLite, той самий обсяг і payload до/після; окремий HTTP client на користувача, без think time; прогрів одним запитом; задана тривалість етапу 3 с. При перевантаженні baseline фактичний етап довший через завершення запитів і timeout 20 с. Результати з timeout позначено як помилки, а p95 тоді описує завершення спроб, не час успішних відповідей. Це короткий локальний дослід, не SLA.

Кеш Fleet виміряно окремо: 1000 повторень того самого repository lookup до/після прогріву TTL cache на тій самій SQLite. Цей результат є часом локального Python-виклику, а не HTTP RTT.

10. Тести, контракти та конвеєр

Domain-тести не потребують SQLite/HTTP. Інтеграційні тести використовують тимчасові SQLite й ASGI. Контракт Trips → Fleet задає незалежні очікування споживача і окрему перевірку постачальника. Актуальні OpenAPI експортуються з виконуваного коду; pipeline відхиляє розбіжність зі збереженим JSON.

Fitness: 0 сторонніх імпортів у домені; 0 циклів; 0 infrastructure-імпортів у application; domain line coverage >=95%; p95<=500 мс при 100 users, 0 помилок і щонайменше 100 відповідей. Динамічний gate запускається окремо без coverage, щоб інструментування не спотворювало результат.

Самостійний пакет має власні run.py, .vscode, .gitignore і .github/workflows/architecture.yml. Інші лабораторні не потрібні для виконання коду, перевірок або відтворення наведених структурних метрик. Це підтверджено ізольованим прогоном окремої копії: strict TypeScript build, 85 функціональних кейсів, 100% domain coverage, HTTP gate, негативний дослід і 9 HTTP smoke-перевірок. Доказ - docs/evidence/standalone.json. ADR-014 фіксує цю межу.

GitHub Actions: checkout → Python 3.13 + Node 24 → pinned requirements → TypeScript build → pipeline → негативний дослід → upload evidence. Невдалий subprocess зупиняє job. Негативний дослід додає import sqlite3 у тимчасову копію domain.py, отримує exit 1 і доводить незмінність оригіналу. Нові регресії захищають конкурентне закріплення ТЗ, повтор пробігу, втрату відповіді, локальний збій між двома комітами та оновлення номера у проєкції.

Аудит за файлами, рівнями, кількістю і фактичним часом - docs/TEST_AUDIT.md. Контрактні й architecture-тести виділено окремо; інтеграційні ASGI-тести не називаються наскрізними. Окремий live_smoke запускає справжні HTTP-процеси.

11. Прийнятий технічний борг

Спрощення	Чому прийнятне зараз	Умова перегляду
SQLite, один worker	Простий автономний навчальний запуск	Конкурентні записи або p95>500 мс на цільовому навантаженні
Локальний TTL cache	Один інстанс власника	Другий інстанс: distributed invalidation/row ACL
Shared HS256, локальний audit	Взаємна довіра процесів на одному ПК	Зовнішній deployment: IdP/JWKS, TLS, захищений audit
Повтор lifecycle вручну після збою	Квитанції роблять повтор безпечним	Потреба безоператорного відновлення: durable coordinator/reconciler
Polling outbox, без retention	Малий обсяг стенду	Ріст журналу/затримки: broker, backoff, retention, метрики
Немає backup drill	Навчальні демонстраційні дані	Реальні дані: резервування й вимір RTO/RPO
GPS/карти/рольове адміністрування відсутні	Обсяг циклу зосереджено на архітектурі	Підтверджена бізнес-потреба і окремі acceptance criteria

12. План еволюції на три кроки

1. Надійне відновлення: durable lifecycle coordinator, outbox retention, backup/restore drill. Тригер - експлуатація без оператора або реальні дані. Критерій: recovery після збою не потребує ручного повтору; вимір RTO/RPO. 2. Промислова межа довіри й зберігання: TLS, IdP/JWKS, PostgreSQL, метрики, квоти. Тригер - доступ поза localhost чи конкуренція записів. Критерій: той самий CI, contract tests і ціль NFR-1 на цільовому середовищі. 3. Зовнішні інтеграції: карти, телематика, сповіщення через адаптери. Тригер - погоджені сценарії NFR-2/3/8. Критерій: окремі навантажувальні й outage-тести, включно з 10 хв недоступності каналу.

13. Висновок

Задум ЛР1 починався модульним монолітом з планом зовнішніх інтеграцій. Фактичний результат - локальна сервісна система з двома власниками даних, перевірюваними бізнес-правилами, ролями, outbox/inbox і вимірною швидкодією. Зміни меж пояснюють ADR, а поточні обмеження мають конкретні тригери. Результати, які не вимірювалися, позначені окремо, без приписування їм успіху.

Джерела

Методичні вказівки ЛР1-ЛР8, надані користувачем; актуальний lab1.docx, зазначений у початковому аналізі репозиторію; вихідний код і історія VladyslavCh412/labs; відтворювані локальні перевірки 03.11.2026. Технічні рішення також спираються на матеріали, наведені у методичках: R. Martin, Clean Architecture; S. Newman, Building Microservices; M. Fowler, Refactoring; M. Kleppmann, Designing Data-Intensive Applications. Книги наведено як рекомендовані методичками джерела, без дослівних цитат.

Додаток А. Аудит і докази

Виміряно 03.11.2026 з JUnit XML. Час містить setup/call/teardown для кейсу; це не лише час функції.

Рівень	Кількість	Сума часу, с	Залежності
architecture	5	0.080	AST та тимчасова копія джерел
integration	40	4.300	Тимчасова SQLite / ASGI / mock transport
unit	31	0.063	Немає БД/HTTP
contract	9	0.175	Незалежний контракт + provider або mock

Окремо: 1 реальний HTTP performance gate (100 users, 0 errors, p95<=500 мс). Live smoke запускає три HTTP-процеси; його 9 перевірок не додаються до кількості pytest-кейсів.

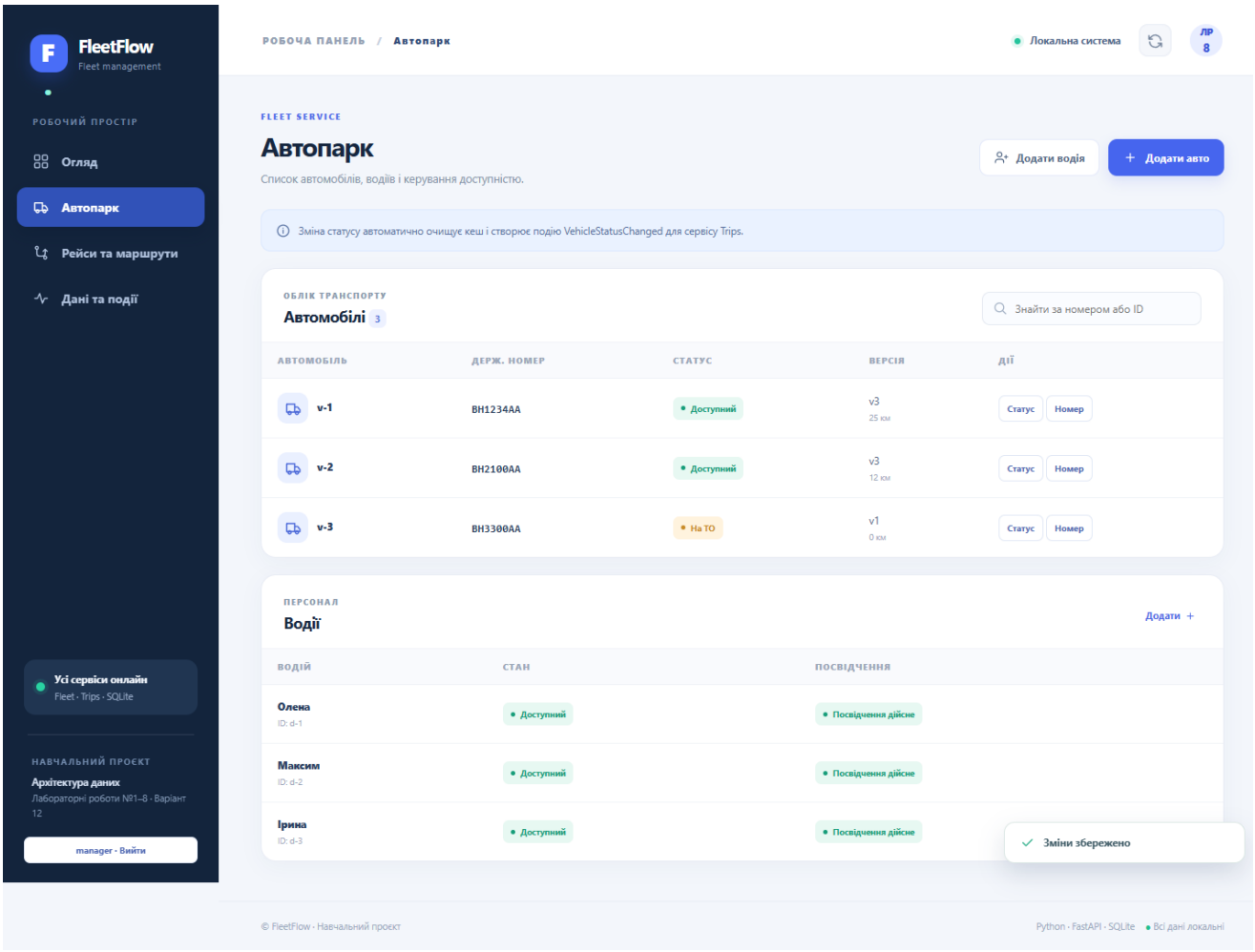
Файл тестів	Рівні	Кількість	Час, с
tests.test_architecture_extensions	architecture, integration	4	0.087
tests.test_characterization	integration, unit	4	0.050
tests.test_contract	contract	9	0.175
tests.test_dashboard	integration	1	0.268
tests.test_domain	unit	21	0.021
tests.test_extension_points	integration, unit	4	0.040
tests.test_fitness	architecture	3	0.068
tests.test_lab3	integration, unit	10	1.333
tests.test_lab5	integration, unit	7	0.386
tests.test_lifecycle	integration, unit	9	0.788
tests.test_pagination	integration	1	0.028
tests.test_read_cache	integration, unit	3	0.136
tests.test_security	integration	9	1.238

Баланс: доменні правила та in-memory порти перевіряються модульно; більшість boundary-кейсів - швидкі інтеграційні ASGI/SQLite, а не повні E2E. Немає потреби піднімати весь стенд для кожного правила. Піраміда не ідеальна за кількістю, але локальна функціональна suite займає кілька секунд; дорогий HTTP gate відокремлено.

Правила ЛР2 → тести: Vehicle identity/mileage/ТО → test_domain.py; Driver.can_drive → test_driver_permission; Route stops → test_route_requires_two_nonempty_stops; Trip lifecycle → test_trip_lifecycle_and_mileage і test_transition_invariants_without_database. Persistence, migration, cache/event replay → test_lab5.py. Конкуренція і відновлення двох власників → test_lifecycle.py.

Негативний дослід: import sqlite3 у копії fleet_service/domain.py → ARCHITECTURE FAILED, returncode=1; original_source_unchanged=true. Live HTTP smoke: усі 9 перевірок пройшли.

Додаток В. Демонстрація застосунку



Демонстраційний рейс завершено на 25 км; пробіг збережений у Fleet. Облікові дані не включено.

Додаток С. Повний журнал ADR

ADR-001

Поточний статус: замінено щодо deployment рішенням ADR-003. Нижче збережено оригінальний контекст і формулювання.

9. ADR-001: Вибір архітектурного стилю Статус: Прийнято (навчальне рішення). Контекст: Потрібно створити систему для невеликого автопарку з обліком ТЗ, водіїв, ТО, рейсів, маршрутів і GPS-даних. У невеликій навчальній команді важливі простота розробки, вимірна продуктивність, надійність та можливість розширення. Розглянуті альтернативи: A) Модульний моноліт. B) Моноліт із виділеним фоновим обробником. C) Набір із 2-3 сервісів. Рішення: Обрати модульний моноліт із чіткими межами модулів. Інтеграції із зовнішніми сервісами ізолювати адаптерами. Передбачити можливість додати чергу та фоновий обробник для телеметрії. Аргументація: Найкращий баланс простоти, цілісності транзакцій і підтримуваності для невеликої команди. Мікросервісний підхід створює зайві витрати на розгортання, моніторинг і мережеву взаємодію до підтвердження потреби в незалежному масштабуванні. Наслідки: Позитивні: простіше розгортання, тестування та транзакції. Негативні: масштабування вебчастини й обробки даних спочатку пов’язане; потрібна дисципліна модульних меж. Ризик контролюється автоматизованими тестами та метриками навантаження. Умови перегляду: Якщо затримка телеметрії перевищує 10 с за цільового навантаження або модулі потребують незалежних релізів, створити новий ADR щодо виділення обробника/сервісу.

ADR-002

10. ADR-002: Вибір технологічного стека Статус: Прийнято для навчального прототипу, відповідність продуктивності підлягає перевірці. Контекст: Для обліку автопарку, маршрутів і рейсів потрібен зрозумілий REST API, реляційне сховище, автоматизовані тести та простий запуск проєкту на комп’ютерах учасників без додаткового сервера СУБД. Розглянуті альтернативи: A) Python + FastAPI + SQLite. B) Python + Django REST Framework + PostgreSQL. C) Java + Spring Boot + PostgreSQL. Рішення: Python 3.12, FastAPI, Pydantic, Uvicorn, SQLite, автоматична специфікація OpenAPI, pytest і Git. Для міграцій бази застосовувати версійовані SQL-скрипти. Після декомпозиції в ЛР3 кожен сервіс використовує власний SQLite-файл. Аргументація: Обрані засоби достатні для навчального веб API, бізнес-правил, тестування та демонстрації взаємодії сервісів. FastAPI забезпечує перевірку схем і автоматичну документацію. SQLite усуває необхідність адміністрування окремого сервера БД, що виправдано для невеликого локального стенду. Компромісом є обмежена масштабованість конкурентних записів. Наслідки: Позитивні - просте налаштування, швидкий запуск, мінімум інфраструктури, зручна перевірка API через /docs. Негативні - SQLite має обмеження під високим паралельним навантаженням, тому результати локального стенду не можна автоматично переносити на промислову систему. Модульні межі та абстракції репозиторіїв спрощують подальшу заміну СУБД. Умови перегляду: Якщо вимірювання покажуть неприйнятну затримку за цільової кількості одночасних користувачів, часті конфлікти запису або вимогу окремого промислового розгортання, створити новий ADR і розглянути серверну СУБД, наприклад PostgreSQL.

ADR-003

8. ADR-003 - Межі сервісів і спосіб взаємодії Поле Рішення Контекст ЛР2 містила одне FastAPI-ядро; необхідні окремі доменні межі, API, події та дослід відмови. Розглянуті альтернативи Один модульний моноліт; Fleet + Trips; 5+ мікросервісів. Прийнято Fleet і Trips як два процеси з власними SQLite-файлами; gateway; синхронний HTTP GET + outbox TripCreated. Наслідки Незалежність даних, слабше зв’язування через події; натомість мережеві відмови, додаткові процеси й відсутність спільної ACID-транзакції. Статус Прийнято для навчальної архітектурної еволюції від ADR-001; не є промисловим розгортанням.

ADR-004

ADR-004 Патерни для наявних залежностей

Дата запису: 2026-10-27. Статус: Прийнято; ретроспективний запис рішень ЛР4.

Контекст: Шлюз містив умовну маршрутизацію; адаптація JSON та створення залежностей були змішані; сповіщення жорстко зв’язані.

Альтернативи: Залишити if/elif і прямі виклики; узагальнити весь застосунок.

Рішення: Strategy, Adapter, Factory, Observer у конкретних проблемних місцях.

Наслідки: Підміна залежностей і адитивне розширення; додаткові класи виправдані регресійними тестами.

ADR-005

ADR-005 Дані та подієва проєкція

Дата запису: 2026-10-27. Статус: Прийнято; ретроспективний запис ЛР5.

Контекст: Потрібні зворотно сумісні зміни номера та читання стану Fleet у Trips.

Альтернативи: Спільна БД; синхронне копіювання; outbox/inbox.

Рішення: Чотири SQL-міграції; cache TTL 30 с; event_id + version; bridge з ACK після успіху.

Наслідки: Кінцева узгодженість; кеш локальний; під час outage верхня межа lag не гарантована.

ADR-006

ADR-006 Токени та права у власників

Дата запису: 2026-10-27. Статус: Прийнято в ЛР6.

Контекст: ЛР5 дозволяла зміни без автентифікації; gateway можна обійти.

Альтернативи: Лише gateway; JWT у кожному сервісі; зовнішній IdP.

Рішення: Локальні Argon2id-облікові дані, HS256 JWT 15 хв і RBAC у сервісах.

Наслідки: Повторна перевірка; взаємна довіра ключу; TLS/IdP потрібні перед deployment.

ADR-007

ADR-007 Обмежена проєкція сторінки рейсів

Дата запису: 2026-10-27. Статус: Прийнято в ЛР6.

Контекст: fetchall матеріалізує весь набір для першої сторінки, що підтверджено Server-Timing.

Альтернативи: Кеш усіх сторінок; LIMIT/OFFSET; нова БД.

Рішення: Обмежити вибірку SQL LIMIT та не дублювати JSON-перетворення готової проєкції.

Наслідки: p95 поліпшився; великий OFFSET і конкурентні записи потребують окремого аналізу.

ADR-008

ADR-008 Архітектурні перевірки у CI

Дата запису: 2026-10-27. Статус: Прийнято в ЛР7.

Контекст: Ручний контроль імпортів і контрактів не запобігає регресії.

Альтернативи: Ручний review; локальні тести; обов’язковий конвеєр.

Рішення: AST, контракт consumer/provider, domain coverage ≥95%, окремий HTTP p95 gate.

Наслідки: Збірка може впасти через повільний runner; поріг не послаблювати, умови дослідити.

ADR-009

ADR-009 Документація фактичної системи

Дата запису: 2026-10-27. Статус: Прийнято в ЛР8.

Контекст: Початковий контекст мав GPS, карти та зовнішні сповіщення, яких немає у прототипі.

Альтернативи: Показувати задум як реалізацію; явно відділити факт і план.

Рішення: Три C4-рівні для поточного коду; таблиця всіх восьми NFR із невиміряними пунктами.

Наслідки: Чесна межа висновків; документацію оновлювати разом із кодом.

ADR-010

ADR-010 Обмежений кеш сторінок Trips

Дата: 2026-10-27. Статус: прийнято. Доповнює ADR-007, не змінює його пороги або початкові вимірювання.

Контекст: у першому GitHub Actions запуску 40492118294 ЛР7 отримала p95 516.807 мс за 100 користувачів проти порогу 500 мс; ЛР8 пройшла на іншому runner. Усі функціональні тести пройшли. Repository p95 ЛР7 становив 216.25 мс: одночасні однакові читання створювали зайві SQLite-з'єднання й конкуренцію.

Альтернативи: послабити поріг (відхилено, суперечить ЛР1); просто повторювати тести до зеленого (не усуває причину); Redis (додаткова інфраструктура); bounded cache з явною інвалідацією.

Рішення: кеш читання сторінок у самому Trips. Ключ (limit, offset, active); TTL 1 секунда; максимум 128 записів; single-flight через Lock при промаху; інвалідація всіх сторінок після успішного create_trip_and_publish. Кеш зберігає лише JSON-проєкцію; він не змінює власника даних. Перевірка токена і ролі відбувається в middleware до звернення до кешу. Усі дозволені ролі зараз бачать однакову проєкцію.

Наслідки: однакові читання не відкривають SQLite на кожен запит. Кеш локальний для процесу; в іншому процесі чи при зовнішній зміні БД дані можуть відставати до 1 секунди. Перед додаванням нових записувальних endpoint або row-level ACL необхідно додати інвалідацію / розширити ключ та перевірки ізоляції. Довгі промахи серіалізуються, але loader обмежений SQL LIMIT і не викликає мережу.

Доказ: tests/test_read_cache.py перевіряє TTL, capacity, конкурентний single-flight та негайну видимість нового рейсу. Поточний набір має 71 перевірку. Перший невдалий хмарний прогін не приховується; результат наступного прогону доступний у Actions.

ADR-011

ADR-011: Збережений життєвий цикл рейсу і повторювані команди

Дата: 2026-11-03. Статус: Прийнято. Доповнює ADR-003/005/010.

Контекст: доменні start/complete були протестовані, але не доступні через HTTP; mileage не зберігався. Розподілений збій може статися після зміни Fleet і до локального commit Trips. Спільна БД порушила б володіння даними.

Альтернативи: залишити лише domain demo; спільна транзакція/БД; 2PC; власні транзакції з durable command receipts і повтором. Обрано останнє.

Рішення: TripUseCases залежить від доменних TripStore/FleetGateway. Fleet виконує start/complete у BEGIN IMMEDIATE: перевіряє ТЗ/водія, зберігає vehicle_usage, outbox і квитанцію trip_id:operation атомарно. Повтор з іншим payload - 409; той самий повтор повертає попередній результат. Trips після успіху Fleet атомарно зберігає свій стан і receipt; його запис захищений перевіркою попереднього стану. Кеші інвалідуються при змінах.

Наслідки: пробіг не подвоюється; ТЗ не займають два рейси; відмова після віддаленого commit відновлюється повтором тієї самої дії. Тимчасова неузгодженість можлива. Durable автоматичний coordinator не реалізовано; для unattended роботи потрібен reconciler з збереженими pending commands.

Докази: test_lifecycle.py - concurrency, payload conflict, lost reply, local failure, persistence, service-role boundary; live_smoke - реальні HTTP. Умова перегляду: production або вимога автоматичного recovery без користувача.

ADR-012

ADR-012: Єдиний відтворюваний пакет ЛР8

Дата: 2026-11-03. Статус: Прийнято. Уточнює ADR-009.

Контекст: README посилався на PDF і PPTX поза Git, а останній commit видалив їх. Користувач запросив готовий проєкт і окрему публікацію ЛР8 у ветці lab8-complete, без підготовки нових звітів ЛР1-ЛР7.

Альтернативи: знову передавати файли окремо; зберегти тільки Markdown; включити до ветки джерела, фінальний PDF/PPTX і санітизовані докази. Обрано останнє. Винятки .gitignore дозволяють лише submission-файли ЛР8.

Рішення: одна точка запуску run.py, налаштування VS Code, актуальні machine-readable OpenAPI, ARCHITECTURE.md з усіма 8 початковими NFR, C4 L1/L2/L3, ADR, TEST_AUDIT, evidence та презентація <=10 хв. Дані, ключі, паролі, venv і робочі БД не комітяться.

Наслідки: clone містить самодостатні матеріали; бінарні PDF/PPTX збільшують Git. Джерела Markdown/діаграм дають змогу переглянути зміст. Результат короткого benchmark не прирівнюється до production SLA. Умова перегляду: регулярні великі медіафайли - перенести у GitHub Releases.

ADR-013

ADR-013: Фронтенд на TypeScript зі строгою перевіркою

Дата: 2026-11-03. Статус: Прийнято. Доповнює ADR-002/012.

Контекст: користувач уточнив обов'язковий стек - TypeScript для фронтенду, FastAPI + SQLite для бекенду. Поточна DOM-панель вже покриває сценарії; зміна UI-фреймворку не потрібна для виконання цієї вимоги.

Альтернативи: переписати на React; перейменувати JS без типізації; типізувати наявний DOM-застосунок і ввести реальну компіляцію TypeScript. Обрано останнє: найменше змін поведінки й перевірювана вимога до мови.

Рішення: frontend/src/app.ts - вихідний код, strict/noEmitOnError, DTO та стан панелі мають типи, API-клієнт використовує generics, DOM-події й помилки звужуються перед доступом. npm lock фіксує TypeScript 7.0.2. FastAPI віддає тільки frontend/build, а не src або node_modules. root setup/start/test/verify та CI виконують збірку. Бекенд залишається FastAPI + власна SQLite кожного сервісу.

Наслідки: помилки типів зупиняють збірку; потрібен Node.js для розробки. Згенерований JS включено як runtime artifact; джерело істини - .ts. Runtime-валідація бізнес-даних залишається у Pydantic на бекенді. Умова перегляду: складніша навігація/компоненти - окремий ADR про фреймворк.

ADR-014

ADR-014: Самостійний кінцевий продукт в одній папці

Дата: 2026-11-03. Статус: Прийнято. Уточнює ADR-012/013.

Контекст: користувачу потрібна тільки підсумкова ЛР8, яка сама містить усі реалізовані результати циклу. Кореневий launcher та посилання на попередні папки робили комплект менш зручним для окремого використання.

Альтернативи: відкривати весь монорепозиторій; дублювати всі старі лабораторні; зробити один самостійний каталог кінцевого продукту. Обрано останнє.

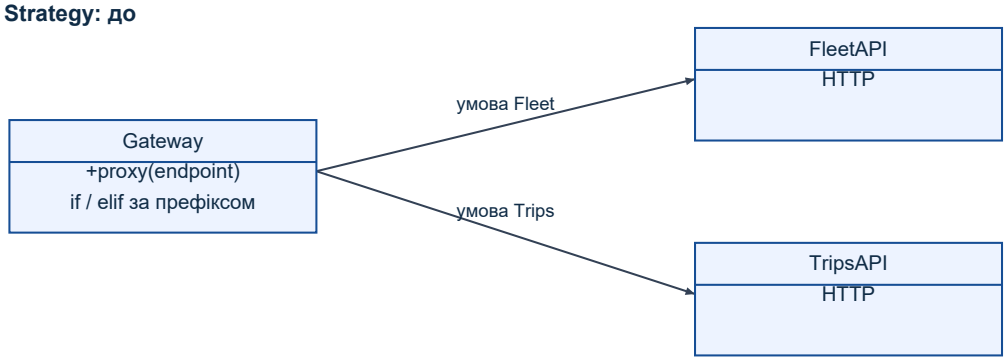
Рішення: run.py, локальна .venv, VS Code, ignore/attributes і standalone CI належать цій папці. Сюди включено before/after для двох патернів, мінімальні before-джерела для структурних вимірювань і справжню історію комітів. Матриця вимог посилається на поточні шари/порти/сценарії та локальні докази. Відокремлена копія без сусідніх лабораторних проходить збірку й pipeline.

Наслідки: каталог можна переносити як окремий проєкт. Історичні файли у docs/refactoring/before не є другим застосунком і не виконуються. Невиміряні початкові NFR лишаються явно позначеним боргом; організаційні ролі команди й усний захист не підміняються програмними твердженнями. Умова перегляду: поділ продукту на окремі репозиторії сервісів.

Додаток D. Патерни: діаграми до і після

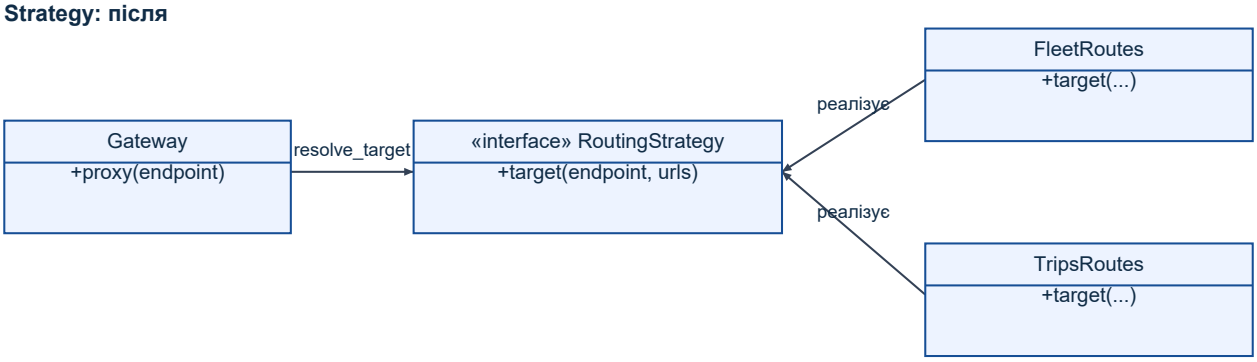
Strategy: залежності центрального proxy

Strategy: до



Класи та залежності; знімок до — commit 46aa488; після — поточний кінцевий продукт

Strategy: після



Класи та залежності; знімок до — commit 46aa488; після — поточний кінцевий продукт

Дві гілки вибору сервісу в proxy замінено вибором RoutingStrategy. Одна гілка перевірки невідомого endpoint лишається. Локальні показники наведено в розділі 6.

Observer: продуцент та підписники

Observer: до



Класи та залежності; знімок до — commit 46aa488; після — поточний кінцевий продукт

Observer: після



Класи та залежності; знімок до — commit 46aa488; після — поточний кінцевий продукт

NotificationObserver реалізує TripCreatedObserver; інші реалізації додаються до списку підписників без зміни TripsRepository. Пряму SQL-вставку notification_inbox перенесено з продуцента до підписника.